

Cahier des charges – Projet de stage développement logiciel

1. Contexte du projet

L'entreprise utilise actuellement ABSSolute comme ERP. Certaines données de tournée existent déjà dans la base SQL liée à ABSSolute ou dans des vues SQL existantes. Aujourd'hui, les livreurs utilisent encore des fiches papier pour suivre leurs tournées et noter certaines informations terrain.

Le problème principal est que le fonctionnement actuel n'est pas assez pratique pour le comptage des chariots, tapis, sacs ou autres éléments liés à la livraison. L'objectif du projet est donc de créer une application mobile simple permettant aux livreurs de consulter leur tournée du jour, de saisir les informations nécessaires pendant la journée, puis de renvoyer les données en fin de tournée.

Le projet doit d'abord se concentrer sur un cas d'usage complet : la prise en charge numérique d'une fiche de tournée quotidienne.

Une évolution importante est également prévue : une interface web interne pour le service expédition. Cette interface permettra de saisir la veille certaines quantités prévues, appelées pré-remplis, afin qu'elles soient proposées au livreur lors du chargement de sa tournée. Ces valeurs resteront modifiables par le livreur sur le terrain, car il peut exister une différence entre ce qui était prévu et la réalité.

Les autres fonctionnalités, comme une interface administrative complète, des exports avancés ou des écrans de suivi plus détaillés, peuvent être prévues dans l'architecture mais développées plus tard si le temps le permet.

2. Objectifs du stage

Le stagiaire aura pour mission de concevoir et développer une solution complète comprenant :

- une application mobile Android pour les livreurs ;
- une API intermédiaire ASP.NET Core ;
- une base SQL Server avec des tables dédiées au projet mobile ;
- une interface web expédition interne ;
- une documentation technique et fonctionnelle ;
- des contrats JSON stables et évolutifs.

3. Périmètre fonctionnel général

La solution devra permettre de :

- identifier le livreur par son code livreur ;
- afficher la date du jour sans permettre sa modification côté livreur ;
- choisir la tournée du jour ;
- charger la tournée depuis l'API ;
- stocker localement la tournée sur le téléphone ;

- consulter les clients ou points de livraison ;
- afficher les informations utiles à la livraison ;
- afficher les instructions permanentes issues d'ABSSolute ;
- afficher les commentaires exceptionnels stockés dans la base du projet mobile ;
- saisir les quantités livrées et récupérées ;
- utiliser des valeurs pré-remplies pour certaines quantités livrées ;
- corriger les saisies tant que la tournée n'a pas été envoyée ;
- envoyer les données en fin de tournée ;
- empêcher le double envoi d'une tournée ;
- verrouiller la tournée côté mobile après synchronisation réussie ;
- conserver les données envoyées dans SQL Server ;
- préparer une exploitation administrative future des données.

4. Application mobile Android

4.1 Rôle de l'application mobile

L'application mobile sera utilisée par les livreurs sur des téléphones Android professionnels. Chaque livreur dispose normalement de son propre téléphone, mais une identification par code livreur reste obligatoire afin d'associer correctement les saisies à la bonne personne.

L'application sera développée avec Visual Studio, probablement en .NET MAUI ou avec une autre solution compatible avec Android. La cible prévue est Android 12.

4.2 Fonctionnalités attendues

L'application devra permettre de :

- identifier le livreur par son code livreur ;
- afficher le nom du livreur après identification ;
- afficher la date du jour en lecture seule ;
- choisir ou confirmer la tournée du jour ;
- charger la tournée du jour depuis l'API ;
- enregistrer la tournée localement sur le téléphone ;
- fonctionner hors connexion pendant la journée ;
- afficher la liste des clients ou points de livraison ;
- afficher le détail d'un point de livraison ;
- afficher les quantités à saisir en deux colonnes : Livré et Récupéré ;
- pré-remplir certaines quantités livrées si l'expédition les a saisies la veille ;
- permettre au livreur de modifier les valeurs pré-remplies ;
- saisir un commentaire livreur si nécessaire ;
- rendre obligatoire le commentaire en cas d'anomalie ou de passage non fait ;
- afficher un récapitulatif avant l'envoi final ;
- envoyer les données en JSON à l'API ;
- bloquer la tournée côté mobile après synchronisation réussie.

4.3 Sécurité d'usage mobile

L'application doit éviter les erreurs d'utilisation terrain.

Le bouton retour Android du téléphone devra être désactivé dans l'application afin d'éviter qu'un livreur perde sa tournée chargée ou revienne involontairement à un écran non prévu.

La navigation devra être maîtrisée par les boutons visibles dans l'application.

En production, il ne sera pas prévu de bouton libre de réinitialisation de la tournée locale. Une telle fonctionnalité serait trop risquée pour un usage livreur. Une procédure technique ou administrative pourra être prévue séparément si nécessaire.

4.4 Stockage local

La tournée chargée devra être stockée localement sur le téléphone, par exemple avec SQLite.

Cela permettra au livreur de continuer à utiliser l'application même sans connexion pendant la journée.

Au lancement de l'application, s'il existe une tournée locale non synchronisée, l'application devra rouvrir cette tournée au lieu de repartir sur un nouveau chargement.

5. API intermédiaire ASP.NET Core

5.1 Rôle de l'API

L'API intermédiaire sera développée en ASP.NET Core. Elle servira de point de communication entre l'application mobile, l'interface web expédition et SQL Server.

Elle jouera un rôle central dans le projet, car elle permettra de :

- éviter une connexion directe entre les téléphones et SQL Server ;
- éviter une connexion directe entre le navigateur expédition et SQL Server ;
- lire les données nécessaires via des vues SQL ;
- récupérer les livreurs, tournées, clients, points de livraison et articles utiles ;
- envoyer au mobile uniquement les données nécessaires ;
- recevoir les saisies effectuées par les livreurs ;
- recevoir ou exposer les pré-remplissages saisis par l'expédition ;
- contrôler les données reçues ;
- transformer les données JSON en données SQL ;
- écrire les résultats dans des tables dédiées ;
- empêcher le double envoi d'une tournée ;
- verrouiller une tournée après synchronisation ;
- centraliser les règles métier ;
- faciliter l'évolution du projet.

Des procédures stockées pourront éventuellement être utilisées plus tard si l'entreprise souhaite centraliser certaines règles métier directement côté SQL Server.

5.2 Routes principales prévues

Les routes principales prévues sont :

- chargement d'une tournée du jour ;

- récupération des tournées disponibles ;
- synchronisation finale d'une tournée ;
- consultation ou création des pré-remplissages expédition ;
- récupération des articles saisissables ;
- contrôle de santé de l'API.

Les routes liées au mobile et les routes liées à l'expédition devront être distinguées lorsque les règles de sécurité sont différentes. Il ne faut cependant pas dupliquer inutilement deux routes qui feraient exactement le même travail. Si une même donnée est utile à plusieurs interfaces, une route commune pourra être utilisée avec une sécurité adaptée.

6. Interface web expédition

6.1 Rôle de l'interface expédition

Une interface web interne sera prévue pour le service expédition.

Elle permettra de saisir la veille les quantités prévues pour certains articles livrés. Ces quantités seront ensuite transmises au mobile lors du chargement de la tournée.

Cette interface ne remplacera pas l'application mobile livreur. Elle servira uniquement à préparer certains pré-remplissages avant le départ des tournées.

6.2 Périmètre de la première version

Pour la première version, l'interface expédition devra permettre uniquement :

- la connexion d'un utilisateur expédition ;
- le choix d'une date de tournée ;
- le choix d'une tournée ;
- l'affichage des clients ou points de livraison de la tournée ;
- la saisie des quantités pré-remplies pour la colonne Livré ;
- l'enregistrement des pré-remplissages ;
- la modification des pré-remplissages tant que la date de blocage n'est pas atteinte ;
- la conservation de l'identité de l'utilisateur ayant créé ou modifié une valeur ;
- la conservation d'un historique des modifications.

L'interface ne devra pas, dans un premier temps, proposer une consultation complète des tournées envoyées. Cette fonctionnalité pourra être prévue plus tard dans une interface administrative.

6.3 Utilisateurs de l'interface expédition

L'interface sera utilisée au départ par un seul poste expédition.

Chaque personne utilisant l'interface devra disposer de son propre compte afin de savoir qui a créé ou modifié les pré-remplissages.

Une authentification simple par identifiant et mot de passe est prévue pour la première version.

6.4 Règles de pré-remplissage

Les pré-remplissages sont saisis par le service expédition la veille de la tournée.

Pour la première version, les articles concernés sont :

- rolls ;
- tapis ;
- sacs.

Le pré-remplissage concerne uniquement la colonne Livré.

La colonne Récupéré reste entièrement saisie par le livreur pendant la tournée.

Le pré-remplissage est optionnel. Un client peut donc ne pas avoir de valeur pré-remplie.

Il faut distinguer clairement :

- valeur vide : l'expédition n'a pas renseigné de valeur ;
- valeur 0 : l'expédition a volontairement prévu zéro.

Le livreur pourra modifier la quantité pré-remplie dans l'application mobile, car la réalité terrain peut être différente de ce qui était prévu.

La valeur prévue par l'expédition et la valeur réellement saisie par le livreur devront être conservées. L'écart pourra ensuite être calculé à l'affichage ou dans un export.

6.5 Blocage des modifications expédition

Les pré-remplissages sont modifiables par l'expédition la veille.

À partir de minuit, les pré-remplissages de la tournée du jour ne devront plus être modifiables par l'interface expédition.

Cette règle permet d'éviter qu'une tournée déjà prête ou bientôt chargée par le livreur soit modifiée trop tardivement.

6.6 Architecture de l'interface web

L'interface web expédition ne doit pas être une simple page JavaScript appelant directement l'API centrale.

L'architecture prévue est :

```

Navigateur du poste expédition
→ application web expédition avec backend serveur
→ API ASP.NET Core centrale
→ SQL Server

```

Le navigateur pourra voir ses échanges avec l'application web expédition, mais il ne devra pas voir directement les échanges internes entre le backend de l'interface web et l'API centrale.

Un éventuel secret d'appel API devra être stocké uniquement côté serveur, jamais dans le JavaScript envoyé au navigateur.

Le choix technique envisagé pour l'interface web est une application ASP.NET Core MVC ou Razor Pages, hébergée sous IIS dans une VM Windows séparée. Ce choix reste cohérent avec l'API ASP.NET Core et l'environnement Windows/IIS.

6.7 Sécurité interface web vers API

L'échange entre l'interface web expédition et l'API devra être sécurisé.

La solution recommandée est :

- HTTPS ;
- restriction IP de la VM interface web vers la VM API ;
- clé API stockée uniquement côté serveur ;
- aucune clé API dans le navigateur ;
- logs des actions importantes.

Les téléphones livreurs ne devront pas pouvoir accéder à l'interface web expédition.

7. SQL Server et stockage des données

7.1 Rôle de SQL Server

SQL Server sera utilisé pour :

- lire les données issues d'ABSSolute via des vues SQL ;
- stocker les données saisies par les livreurs ;
- stocker les pré-remplissages expédition ;
- stocker les commentaires exceptionnels ;
- conserver les informations de synchronisation ;
- conserver les logs techniques et métier ;
- préparer l'exploitation administrative des données.

Les données mobiles ne devront pas modifier directement les tables ABSSolute.

L'approche retenue est d'utiliser des tables dédiées au projet mobile.

7.2 Accès aux données ABSSolute

L'accès aux données ABSSolute se fera via des vues SQL.

Cette approche permet de :

- ne pas dépendre directement des tables internes ABSSolute ;
- limiter les risques de modification accidentelle ;

- simplifier les données envoyées à l'application mobile ;
- centraliser les filtres nécessaires ;
- rendre l'API plus stable.

Lecture des vues ABSSolute :

Serveur : SRVSQL\SQLEXPRESS,58734
Base : lavinprosli

7.3 Base dédiée au projet mobile

Les données saisies par les livreurs, les pré-remplissages et les données propres au projet mobile seront stockés dans une base dédiée.

Base de test projet mobile :

Serveur : SRVSQL\SQL2017
Base : bd_eric

L'API utilisera un compte SQL dédié avec des droits limités :

- lecture seule sur les vues issues d'ABSSolute ;
- écriture uniquement sur les tables du projet mobile ;
- aucun accès direct depuis les téléphones ;
- aucun accès direct depuis le navigateur expédition.

7.4 Tables dédiées au projet mobile

Les tables déjà prévues ou existantes sont :

- Mobile_Livreur : livreurs utilisant l'application ;
- Mobile_ChargementTournee : historique des chargements de tournée ;
- Mobile_Tournee : en-tête de tournée mobile, statut de synchronisation et verrouillage ;
- Mobile_TourneeLigne : lignes clients ou points de passage ;
- Mobile_TourneeLigneQuantite : quantités par article, avec colonnes livré et récupéré ;
- Mobile_LogSynchronisation : logs de chargement, synchronisation, erreur et double envoi ;
- Mobile_ExportAdmin : suivi éventuel des exports.

Des tables supplémentaires devront être prévues pour les nouveaux besoins :

- pré-remplissages expédition ;
- historique des modifications des pré-remplissages ;
- commentaires exceptionnels par date et par client ou point de livraison ;
- utilisateurs de l'interface expédition si l'authentification est gérée dans la base projet.

Exemples de tables possibles :

- Mobile_PreRemplissageTournee ;
- Mobile_PreRemplissageQuantite ;

- Mobile_PreRemplissageHistorique ;
- Mobile_CommentaireExceptionnel ;
- Mobile_UtilisateurExpedition.

Les noms exacts pourront être ajustés lors de la conception technique, mais le besoin fonctionnel doit être prévu dès le cahier des charges.

8. Fonctionnement actuel

Aujourd'hui :

- les données de tournée existent dans ABSSolute ou dans une base SQL Server liée à ABSSolute ;
- les fiches de tournée sont utilisées sous forme papier ou Excel ;
- les livreurs suivent leur tournée manuellement ;
- les informations de terrain, comme le nombre de chariots, doivent être notées ou recomptées ;
- certains éléments prévus sont actuellement renseignés par l'expédition sur les feuilles physiques ;
- l'administration doit ensuite récupérer ou retraiter les informations.

9. Fonctionnement attendu avec la solution

9.1 La veille de la tournée

Le service expédition prépare les tournées.

Depuis l'interface web expédition, les utilisateurs habilités peuvent saisir des quantités pré-remplies pour certains articles livrés.

Pour la première version, les articles concernés sont rolls, tapis et sacs.

Les pré-remplissages restent optionnels.

Les modifications sont possibles uniquement avant minuit pour la tournée du lendemain.

Chaque création ou modification devra être liée à un utilisateur expédition et historisée.

9.2 Début de journée

Le livreur ouvre l'application mobile au dépôt.

Chaque livreur dispose normalement de son propre téléphone, mais il doit tout de même s'identifier par code livreur pour éviter toute erreur d'association des données.

L'application affiche la date du jour. Cette date est non modifiable côté livreur.

Le livreur choisit uniquement sa tournée du jour, puis l'application appelle l'API pour récupérer les données utiles.

La tournée chargée contient :

- les données issues des vues ABSolute ;
- les instructions permanentes issues du champ INSTRUCTIONS de v_tournee ;
- les commentaires exceptionnels éventuels stockés dans la base du projet mobile ;
- les articles saisissables ;
- les quantités pré-remplies éventuelles saisies par l'expédition.

La tournée est ensuite enregistrée localement sur le téléphone afin que le livreur puisse travailler pendant la journée sans dépendre d'une connexion permanente.

9.3 Pendant la tournée

Le livreur consulte la liste des clients ou points de livraison à traiter.

Pour chaque client, il peut voir :

- le numéro client ;
- le nom client ;
- le point de livraison ;
- la description du point de livraison ;
- la zone ;
- la tournée ;
- les instructions permanentes si elles existent ;
- le commentaire exceptionnel si celui-ci existe ;
- le type de linge, vêtement ou article si l'information est disponible ;
- les informations de retour si nécessaire ;
- la zone de déchargement calculée selon la règle métier définie.

Sur la page "Points de livraison", les instructions permanentes et commentaires exceptionnels devront être affichés en bas du bandeau client uniquement s'ils existent.

Si un client n'a ni instruction permanente ni commentaire exceptionnel, aucun bloc supplémentaire ne doit être affiché afin de ne pas surcharger l'interface.

Le livreur peut ensuite saisir les informations terrain, notamment les quantités livrées et récupérées pour chaque type d'article prévu.

L'écran de détail devra présenter deux colonnes distinctes :

- Livré ;
- Récupéré.

Les valeurs saisies seront des entiers positifs ou nuls.

Les quantités négatives sont interdites.

Les retours doivent être gérés dans une colonne séparée, et non par une valeur négative.

Les données sont conservées localement sur le téléphone jusqu'à l'envoi final.

Tant que la tournée n'est pas envoyée, le livreur doit pouvoir corriger ses saisies.

9.4 Fin de journée

En fin de journée, le livreur accède à un écran récapitulatif avant l'envoi.

Cet écran doit lui permettre de vérifier :

- le nombre de clients saisis ;
- les anomalies ;
- les passages non faits ;
- les éventuels commentaires obligatoires manquants.

Avant l'envoi définitif, l'application devra afficher un message clair indiquant qu'une fois les données envoyées, la tournée ne pourra plus être modifiée depuis l'application mobile.

L'application envoie les données en JSON à l'API.

L'API vérifie les données, évite les doublons, puis insère les résultats dans SQL Server.

Après synchronisation réussie, les données sont verrouillées côté mobile.

Les corrections éventuelles relèvent uniquement d'un administrateur ou d'un responsable habilité, hors application mobile livreur, avec traçabilité.

10. Contraintes d'ergonomie mobile

L'application doit rester simple, car elle sera utilisée sur le terrain par des livreurs qui doivent saisir rapidement les informations.

Principes à respecter :

- limiter au maximum les informations à saisir ;
- identifier le livreur par code, puis afficher son nom ;
- ne pas laisser le livreur modifier la date de tournée ;
- laisser le livreur choisir uniquement sa tournée ;
- utiliser des champs numériques simples pour les quantités ;
- afficher un clavier numérique ;
- ne pas utiliser de boutons plus et moins pour les quantités ;
- présenter les quantités en deux colonnes : Livré et Récupéré ;
- permettre la correction des erreurs tant que la tournée n'a pas été envoyée ;
- ne plus permettre de modification une fois l'envoi définitif confirmé ;
- afficher un avertissement avant l'envoi final ;
- désactiver le bouton retour Android pour éviter les sorties ou retours accidentels ;
- ne pas afficher de bouton de réinitialisation libre en production.

11. Architecture globale

L'architecture retenue sépare clairement les responsabilités.

Application mobile Android livreur
→ API ASP.NET Core
→ SQL Server

Pour l'interface expédition :

Navigateur du poste expédition
→ Backend interface web expédition
→ API ASP.NET Core
→ SQL Server

L'application mobile se concentre sur l'usage terrain : consulter, saisir et envoyer.

L'interface web expédition se concentre sur la préparation des pré-remplissages la veille.

L'API centralise la logique métier : contrôle, transformation, sécurité, communication avec SQL Server et gestion de la synchronisation.

SQL Server conserve les données proprement.

Les exports ou une interface administrative complète peuvent être ajoutés plus tard sans modifier toute l'application mobile.

12. Données utilisées

12.1 Données récupérées depuis ABSSolute ou SQL Server

Les données de départ correspondent aux informations de tournée déjà existantes.

Champ	Rôle dans l'application
NUM_CLI	Identifiant du client
NOM_CLI	Nom du client
PDL	Point de livraison
PDL_DESC	Description du point de livraison
JOUR_TOURNEE	Jour de tournée
SCHEMA_LIV	Schéma de livraison
TOURNEE	Code tournée
TOURNEE_DESC	Nom de la tournée, ne détermine pas l'ordre réel
ARRET	Ordre des clients, croissant
JOUR_TOURNEE_RETOUR	Jour de tournée retour, utilisé dans la règle de zone de déchargement

Champ	Rôle dans l'application
TOURNEE_RET	Code tournée retour
TOURNEE_RETOUR_DESC	Description du retour
INSTRUCTIONS	Instructions permanentes du client
ZONE_DECH	Zone de déchargement
DATEINACTIVE	Déjà utilisé comme filtre, non traité directement par l'application
Zone	Zone ou regroupement logistique

Ces données servent principalement à préremplir l'application mobile pour éviter que le livreur ne ressaisisse des informations déjà connues.

Les articles saisissables, tels que les rolls, tapis et sacs, seront gérés dans la base du projet mobile ou côté API pour la première version. Les vues ABSolute sont utilisées en lecture, mais les données propres au projet mobile seront stockées dans la base dédiée.

12.2 Données saisies ou modifiées par l'expédition

Donnée	Règle
Date de tournée	Choisie dans l'interface expédition
Code tournée	Choisi dans l'interface expédition
Client ou point de livraison	Issu de la tournée
Article	Rolls, tapis, sacs pour la première version
Quantité livrée prévue	Optionnelle, entier positif ou nul
Valeur vide	Signifie non renseigné
Valeur 0	Signifie zéro prévu volontairement
Utilisateur de création	Enregistré
Date de création	Enregistrée
Utilisateur de modification	Enregistré
Date de modification	Enregistrée
Historique des modifications	Conservé

12.3 Données saisies sur mobile par le livreur

Donnée	Statut précisé
Code livreur	Saisi ou sélectionné au démarrage, puis mémorisé sur le téléphone
Tournée choisie	Associée au livreur au début de journée, choix manuel possible
Date de tournée	Date du jour affichée mais non modifiable

Donnée	Statut précisé
Quantité de rolls livrés	Entier positif ou nul
Quantité de rolls récupérés	Entier positif ou nul
Quantité de tapis livrés	Entier positif ou nul
Quantité de tapis récupérés	Entier positif ou nul
Quantité de sacs livrés	Entier positif ou nul
Quantité de sacs récupérés	Entier positif ou nul
Autres articles futurs	Structure prévue pour ajout ultérieur sans refonte complète
Commentaire livreur	Saisie possible pour cas particulier, obligatoire si anomalie ou non fait
Statut du passage	À faire, fait, non fait, anomalie
Heure de validation	Enregistrée automatiquement
Identifiant du livreur	Associé à chaque validation

13. Utilisateurs et rôles

13.1 Livreurs

Environ une vingtaine de livreurs utiliseront l'application.

Chaque livreur dispose normalement de son propre téléphone, mais une identification par code livreur reste nécessaire pour fiabiliser les données.

Chaque livreur est normalement affecté à une tournée. Cependant, une tournée peut être reprise par un autre livreur en cas d'absence, maladie ou remplacement. Le choix de la tournée doit donc rester possible après identification.

Parcours livreur :

```

Le livreur s'identifie par code livreur
↓
L'application affiche son nom et la date du jour non modifiable
↓
Il choisit ou confirme sa tournée
↓
Il charge la tournée du jour
↓
Il saisit les quantités client par client
↓
Il vérifie le récapitulatif
↓

```

Il envoie la tournée en fin de journée
↓
La tournée est verrouillée côté mobile

13.2 Utilisateurs expédition

Les utilisateurs expédition devront pouvoir :

- se connecter à l'interface web expédition ;
- choisir une date ;
- choisir une tournée ;
- consulter les clients ou points de livraison de la tournée ;
- saisir les quantités livrées prévues ;
- modifier les quantités pré-remplies avant minuit ;
- sauvegarder les pré-remplissages ;
- être identifiés dans l'historique des modifications.

13.3 Administrateur

L'administrateur doit pouvoir, dans une évolution future :

- consulter les données envoyées ;
- vérifier si toutes les tournées ont été envoyées ;
- relancer manuellement les livreurs si une tournée n'a pas été validée ;
- exporter les données au format CSV ou JSON ;
- consulter les écarts entre prévu et réel ;
- corriger une tournée envoyée si nécessaire, uniquement côté administration et avec traçabilité ;
- gérer les commentaires exceptionnels par date et par client ;
- éventuellement accéder plus tard à une interface d'administration complète.

13.4 Responsable informatique

Le responsable informatique de l'entreprise devra pouvoir :

- maintenir l'API ;
- maintenir l'interface web expédition ;
- redémarrer les services si nécessaire ;
- surveiller le fonctionnement ;
- gérer les accès SQL ;
- gérer les certificats HTTPS si nécessaire ;
- faire évoluer le projet après le stage.

14. Règles métier

14.1 Validation côté livreur

Pour la première version, il n'est pas prévu d'imposer trop de champs obligatoires au livreur afin de ne pas complexifier l'usage terrain.

Les contrôles principaux porteront sur :

- l'identification obligatoire par code livreur ;
- l'affichage de la date du jour en lecture seule ;
- le choix obligatoire d'une tournée ;
- l'impossibilité de saisir des quantités négatives ;
- l'utilisation d'entiers pour les quantités ;
- la séparation entre quantités livrées et quantités récupérées ;
- l'enregistrement automatique de l'heure de validation ;
- l'association de la saisie au livreur ;
- l'association de la saisie à la tournée ;
- la gestion du statut du passage ;
- l'obligation d'un commentaire en cas de statut anomalie ou non fait.

Le livreur peut corriger ses saisies avant l'envoi final.

Après synchronisation réussie, la tournée devient non modifiable côté mobile.

14.2 Validation côté API

L'API devra valider les données avant toute insertion définitive dans SQL Server.

Elle devra notamment :

- refuser une requête sans code livreur ;
- refuser une requête sans code tournée ;
- refuser une requête sans date de tournée ;
- refuser une ligne sans identifiant de ligne source ;
- refuser les quantités négatives ;
- refuser les quantités non entières ;
- refuser une ligne avec statut non fait ou anomalie sans commentaire ;
- refuser une ligne validée sans heure de validation ;
- refuser une synchronisation finale contenant encore des lignes à faire, sauf choix métier contraire validé ;
- refuser un double envoi avec le même identifiant de synchronisation ;
- refuser un double envoi pour la même date, la même tournée et le même livreur.

14.3 Validation côté expédition

L'interface expédition devra valider :

- l'identification de l'utilisateur ;
- le choix d'une date ;
- le choix d'une tournée ;
- l'utilisation d'entiers positifs ou nuls ;
- la distinction entre valeur vide et valeur 0 ;
- l'interdiction de modification après minuit pour la tournée du jour ;
- l'enregistrement de l'utilisateur ayant créé ou modifié les données.

14.4 Instructions permanentes et commentaires exceptionnels

Il existe deux types d'informations à afficher au livreur :

- les instructions permanentes ;
- les commentaires exceptionnels.

Les instructions permanentes proviennent du champ INSTRUCTIONS de la vue v_tournee.

Les commentaires exceptionnels seront stockés dans la base du projet mobile. Ils seront optionnels, liés à une date et à un client ou point de livraison. Ils seront saisis ou modifiés par l'administration, hors ABSolute.

Dans le JSON de chargement, il faudra prévoir :

- un champ pour l'instruction permanente ;
- un champ optionnel pour le commentaire exceptionnel.

Sur l'application mobile, ces informations devront être affichées en bas du bandeau client sur la page "Points de livraison", uniquement si elles existent.

Si aucune instruction permanente et aucun commentaire exceptionnel n'existent pour un client, aucun bloc commentaire ne doit être affiché.

14.5 Pré-remplissages expédition

Les pré-remplissages sont des quantités prévues saisies par le service expédition.

Ils concernent uniquement la colonne Livré.

Ils sont facultatifs.

Ils peuvent être différents des quantités réellement saisies par le livreur.

La quantité prévue et la quantité réelle devront être conservées séparément.

L'écart sera calculé à l'affichage ou dans un export, selon la formule :

$$\text{écart} = \text{quantité réelle livrée} - \text{quantité livrée prévue}$$

14.6 Clients fermés

Les clients fermés pourront être gérés par un filtre côté application.

Deux affichages seront possibles :

- masquer les clients fermés ;
- afficher les clients fermés avec un statut spécifique.

Cette solution permet de laisser de la souplesse à l'administration et aux utilisateurs sans supprimer définitivement l'information de la tournée.

15. Synchronisation et stockage local

L'application mobile devra stocker les données localement pendant la journée, par exemple avec une base locale SQLite.

Cela permet au livreur de continuer à utiliser l'application même si la connexion réseau n'est pas disponible pendant la tournée.

15.1 Statuts de synchronisation

L'application devra prévoir des statuts simples :

- non envoyé ;
- envoyé ;
- erreur d'envoi.

15.2 Échec d'envoi

Si l'envoi échoue le soir, une vérification manuelle sera effectuée par le responsable logistique ou informatique.

Au début du projet, les feuilles papier seront conservées en parallèle afin d'éviter les pertes d'information pendant la phase de transition.

L'application ne permettra pas un nouvel envoi libre d'une tournée déjà synchronisée.

En cas d'échec d'envoi, les données resteront stockées localement et un renvoi exceptionnel pourra être réalisé uniquement si la tournée n'a pas déjà été enregistrée côté serveur.

15.3 Double envoi

La protection contre les doubles envois devra être gérée côté API.

L'API devra vérifier si une tournée du même chauffeur, à la même date, avec le même code tournée, a déjà été envoyée.

Exemple de comportement attendu :

API → Tournée déjà synchronisée
Mobile → Déjà envoyée le 27/04 à 16h12

15.4 Modification après synchronisation

Après synchronisation réussie, la tournée sera bloquée côté livreur.

Une tournée envoyée ne pourra pas être corrigée par le livreur depuis l'application mobile.

Les corrections éventuelles seront réservées à l'administrateur ou à un responsable habilité, dans un outil d'administration, dans un export contrôlé ou directement dans les tables dédiées selon la procédure choisie par l'entreprise.

Ces corrections devront être tracées.

Avant l'envoi final, l'application devra afficher un message de confirmation indiquant que les données ne seront plus modifiables après synchronisation.

16. Contrat JSON

Le contrat JSON doit rester stable et évolutif.

Les ajouts liés aux pré-remplissages et aux commentaires exceptionnels doivent être optionnels afin de ne pas casser le fonctionnement existant.

16.1 Chargement de la tournée du jour

Route recommandée :

```
GET /api/tournees/jour?dateTournée=2026-05-04&codeTournée=2001&codeLivreur=2
```

La date est envoyée par l'application ou calculée côté application à partir de la date du jour, mais elle n'est pas modifiable par le livreur dans l'interface.

Exemple de JSON de chargement enrichi :

```
{
  "schemaVersion": "1.2",
  "dateTournée": "2026-05-04",
  "dateModifiable": false,
  "codeTournée": "2001",
  "libelleTournée": "MDR VENDEE",
  "livreur": {
    "codeLivreur": "2",
    "nomLivreur": "DAVID LEBAS"
  },
  "articlesSaisissables": [
    { "codeArticle": "ROLLS", "libelle": "Rolls" },
    { "codeArticle": "TAPIS", "libelle": "Tapis" },
    { "codeArticle": "SACS", "libelle": "Sacs" }
  ],
  "lignes": [
    {
      "idLigneSource": "2026-05-04|2001|1|1058|PDL01|1",
      "ordreArret": 1,
      "numClient": "1058",
      "nomClient": "HOTEL EXEMPLE",
    }
  ]
}
```

```

"codePDL": "PDL01",
"descriptionPDL": "Entrée principale",
"instructions": "Livraison par l'arrière",
"commentaireExceptionnel": "Prévoir passage avant 10h ce jour-là.",
"quantitesPreRemplies": [
  {
    "codeArticle": "ROLLS",
    "libelle": "Rolls",
    "quantiteLivreePrevue": 2
  },
  {
    "codeArticle": "TAPIS",
    "libelle": "Tapis",
    "quantiteLivreePrevue": null
  },
  {
    "codeArticle": "SACS",
    "libelle": "Sacs",
    "quantiteLivreePrevue": 0
  }
]
}
]
}

```

Dans cet exemple :

- ROLLS avec valeur 2 signifie que l'expédition a prévu 2 rolls livrés ;
- TAPIS avec valeur null signifie que l'expédition n'a pas renseigné de pré-rempli ;
- SACS avec valeur 0 signifie que l'expédition a volontairement prévu zéro sac livré.

16.2 Synchronisation de fin de journée

Route recommandée :

`POST /api/synchronisations`

Exemple de JSON de synchronisation enrichi :

```

{
  "schemaVersion": "1.2",
  "idSynchronisation": "7d3b3d5a-8dc4-4b2c-9f20-6b2170f1b321",
  "dateTournee": "2026-05-04",
  "codeTournee": "2001",
  "libelleTournee": "MDR VENDEE",
  "livreur": {
    "codeLivreur": "2",
    "nomLivreur": "DAVID LEBAS"
  },

```

```

"mobile": {
  "nomAppareil": "Samsung A12",
  "versionApplication": "1.0.0",
  "dateChargement": "2026-05-04T07:15:00",
  "dateEnvoi": "2026-05-04T17:45:00"
},
"commentaireGlobal": null,
"lignes": [
  {
    "idLigneSource": "2026-05-04|2001|1|1058|PDL01|1",
    "ordreArret": 1,
    "numClient": "1058",
    "nomClient": "HOTEL EXEMPLE",
    "codePDL": "PDL01",
    "descriptionPDL": "Entrée principale",
    "instructions": "Livraison par l'arrière",
    "commentaireExceptionnel": "Prévoir passage avant 10h ce jour-là.",
    "statutPassage": "FAIT",
    "estValidee": true,
    "heureValidation": "2026-05-04T15:42:00",
    "commentaireLivreur": null,
    "quantites": [
      {
        "codeArticle": "ROLLS",
        "libelle": "Rolls",
        "quantiteLivreePrevue": 2,
        "quantiteLivree": 3,
        "quantiteRecuperee": 1
      },
      {
        "codeArticle": "TAPIS",
        "libelle": "Tapis",
        "quantiteLivreePrevue": null,
        "quantiteLivree": 4,
        "quantiteRecuperee": 3
      },
      {
        "codeArticle": "SACS",
        "libelle": "Sacs",
        "quantiteLivreePrevue": 0,
        "quantiteLivree": 1,
        "quantiteRecuperee": 0
      }
    ]
  }
]
}

```

Cette structure permet de conserver :

- la quantité livrée prévue par l'expédition ;

- la quantité livrée réellement saisie par le livreur ;
- la quantité récupérée réellement saisie par le livreur.

L'écart entre prévu et réel pourra être calculé ensuite dans un export ou une future interface administrative.

Cette structure est volontairement évolutive : si l'entreprise ajoute plus tard des vêtements, draps, serviettes ou d'autres articles, l'API pourra ajouter un nouvel objet dans le tableau quantités sans modifier entièrement le contrat JSON.

17. Hébergement de l'API

L'API sera hébergée dans une VM Windows située sur le même serveur physique que SQL Server, si les ressources du serveur le permettent.

Cette solution permet :

- d'isoler l'API de SQL Server ;
- d'éviter d'installer directement l'application sur le serveur SQL ;
- de redémarrer l'API sans toucher directement à SQL Server ;
- de rester dans l'infrastructure interne de l'entreprise ;
- de faciliter la maintenance ;
- de limiter les coûts supplémentaires.

17.1 Mode d'exécution de l'API

Le choix recommandé est d'héberger l'API ASP.NET Core sous IIS dans la VM Windows.

IIS est inclus avec Windows Server. Il ne devrait donc pas générer de coût supplémentaire si l'entreprise dispose déjà d'une licence Windows Server.

Le développement pourra se faire localement avec Visual Studio ou en console, mais la version exploitable par l'entreprise devra être déployée sous IIS.

Éléments à prévoir :

- VM Windows ;
- IIS activé ;
- .NET Hosting Bundle installé ;
- API ASP.NET Core publiée ;
- configuration du site ou de l'application dans IIS ;
- accès SQL Server configuré ;
- logs de fonctionnement ;
- procédure de redémarrage documentée.

18. Hébergement de l'interface web expédition

L'interface web expédition sera hébergée sur une VM distincte de la VM API.

Le choix envisagé est :

- application ASP.NET Core MVC ou Razor Pages ;
- hébergement sous IIS ;
- accès réservé au poste expédition ou à certaines adresses IP ;
- communication serveur à serveur avec l'API centrale ;
- aucun appel direct du navigateur vers l'API centrale ;
- aucun secret stocké côté navigateur.

Cette séparation permet de limiter les accès, d'éviter d'exposer l'API directement à l'interface navigateur, et de garder une architecture plus propre.

19. Accès réseau à l'API

La solution la plus adaptée au besoin est un accès à l'API via le Wi-Fi de l'entreprise au dépôt.

Fonctionnement :

Matin :
Téléphone connecté au Wi-Fi du dépôt
→ chargement de la tournée

Pendant la journée :
Utilisation hors connexion
→ stockage local sur le téléphone

Soir :
Téléphone connecté au Wi-Fi du dépôt
→ envoi des données vers l'API

Avantages :

- éviter d'exposer l'API sur Internet ;
- simplifier la sécurisation ;
- correspondre au fonctionnement prévu du projet ;
- éviter de dépendre de la 4G ou de la 5G pendant la tournée ;
- limiter la complexité réseau pour une première version ;
- réduire le besoin de cartes SIM pour les téléphones livreurs ;
- utiliser un Wi-Fi déjà fonctionnel dans le dépôt.

Pour la version finale exploitable en entreprise, l'API devra utiliser HTTPS.

20. Sécurité réseau cible

L'architecture réseau souhaitée est la suivante :

Téléphones Android → API : autorisé en HTTPS
Téléphones Android → SQL Server : interdit

Téléphones Android → interface web expédition : interdit

Poste expédition → interface web expédition : autorisé

Postes non autorisés → interface web expédition : interdit si possible

VM interface web expédition → API : autorisé en HTTPS

VM API → SQL Server : autorisé

Internet → API : interdit

Internet → interface web expédition : interdit

La sécurité exacte dépendra des possibilités réseau de l'entreprise : pare-feu, VLAN, restrictions IP, DNS interne, certificats HTTPS et règles entre VM.

21. Sauvegarde

Des sauvegardes sont déjà prévues dans l'environnement SQL Server.

Le rythme indiqué est de plusieurs sauvegardes par jour, dont une sauvegarde toutes les demi-heures.

Le projet devra respecter ce fonctionnement existant.

Les nouvelles tables du projet mobile devront être incluses dans les sauvegardes, notamment :

- données de tournées envoyées ;
- lignes de tournée ;
- quantités par article ;
- logs ;
- pré-remplissages expédition ;
- historique des modifications ;
- commentaires exceptionnels.

22. Logs

L'API devra journaliser les événements importants :

- chargement d'une tournée ;
- réception d'une synchronisation ;
- synchronisation réussie ;
- erreur d'envoi ;
- tentative de double envoi ;
- erreurs SQL ;
- chargement des pré-remplissages ;
- erreurs de validation ;
- appels sensibles liés à l'expédition.

L'interface expédition devra également conserver les actions importantes :

- connexion utilisateur ;
- création d'un pré-remplissage ;

- modification d'un pré-remplissage ;
- tentative de modification après la date de blocage ;
- erreur d'enregistrement.

23. Exigences de développement

23.1 Qualité du code

Le code devra respecter les principes suivants :

- nommage clair et cohérent ;
- séparation des responsabilités ;
- découpage propre entre interface, logique métier et accès aux données ;
- commentaires utiles ;
- respect des bonnes pratiques ;
- gestion claire des erreurs ;
- structure évolutive ;
- contrats JSON documentés ;
- préparation de l'ajout futur de nouveaux articles ;
- préparation de futures interfaces d'administration.

Les commentaires devront expliquer les choix techniques ou métier, et non décrire des évidences.

23.2 Réutilisabilité

Le système devra être conçu pour pouvoir :

- être adapté à d'autres cas d'usage proches ;
- être maintenu facilement ;
- permettre l'ajout de nouveaux champs sans refonte complète ;
- permettre l'ajout de nouveaux articles dans les quantités ;
- isoler autant que possible la logique métier spécifique ;
- réutiliser la même base technique pour d'autres formulaires mobiles ;
- permettre la création future d'une interface administrative plus complète.

24. Documentation attendue

La documentation devra contenir :

- un README technique ;
- un schéma d'architecture global ;
- un schéma de base de données ;
- une documentation des routes API ;
- une documentation Swagger/OpenAPI ;
- une explication simple du déploiement ;
- une procédure de redémarrage ou de maintenance de l'API ;
- une procédure de maintenance de l'interface web expédition ;
- des exemples de requêtes JSON ;
- une description du fonctionnement de la synchronisation ;
- une description de la structure Livré/Récupéré pour les quantités ;
- une description des pré-remplissages expédition ;

- une description des commentaires exceptionnels ;
- une description des règles de sécurité réseau attendues.

25. Tests automatiques et tests manuels

Des tests automatiques pourront être prévus lorsque le premier prototype fonctionnera.

Les tests devront prioritairement couvrir :

- les routes principales de l'API ;
- la validation des données reçues ;
- l'identification par code livreur ;
- la date non modifiable côté mobile et cohérente côté API ;
- la protection contre les doubles envois ;
- l'insertion SQL ;
- l'insertion des quantités par article ;
- le stockage des quantités pré-remplies ;
- la distinction entre 0 et valeur non renseignée ;
- la conservation du prévu et du réel ;
- le verrouillage après synchronisation ;
- les cas d'erreur de synchronisation ;
- le blocage des modifications expédition après minuit ;
- l'affichage conditionnel des instructions et commentaires exceptionnels.

Des tests manuels devront aussi être réalisés avec un téléphone Android physique connecté au Wi-Fi du dépôt.

26. Conclusion

L'architecture retenue repose sur une application mobile Android, une API ASP.NET Core hébergée dans une VM Windows sous IIS, une interface web expédition hébergée dans une VM séparée, et SQL Server comme base centrale.

Le choix prioritaire est de faire fonctionner l'application mobile en mode principalement hors ligne pendant la tournée, avec chargement et envoi des données depuis le dépôt via le Wi-Fi interne de l'entreprise.

Les ajouts liés à l'expédition permettent de mieux préparer les tournées la veille, sans casser le fonctionnement mobile existant. Les quantités pré-remplies sont optionnelles, concernent uniquement la colonne Livré, et restent modifiables par le livreur sur le terrain.

La solution distingue clairement :

- les données permanentes issues d'ABSolute ;
- les commentaires exceptionnels propres au projet mobile ;
- les quantités prévues par l'expédition ;
- les quantités réellement saisies par le livreur ;
- les corrections ou exploitations administratives futures.

Cette approche permet de répondre au besoin métier sans complexifier inutilement l'usage terrain, tout en gardant une base technique durable, sécurisée et évolutive.